Market_Basket_Analysis

September 9, 2026

[2]:
```
!pip install pandas numpy matplotlib scikit-learn mlxtend openpyxl
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in
.\AppData\Roaming\Python\Python314\site-packages (3.0.5)
Requirement already satisfied: numpy in
.\AppData\Roaming\Python\Python314\sitepackages (2.5.2)
Requirement already satisfied: matplotlib in
.\AppData\Roaming\Python\Python314\site-packages (3.11.1)
Requirement already satisfied: scikit-learn in
.\AppData\Roaming\Python\Python314\site-packages (1.9.0)
Requirement already satisfied: mlxtend in
.\AppData\Roaming\Python\Python314\site-packages (0.25.0)
Requirement already satisfied: openpyxl in
.\AppData\Roaming\Python\Python314\site-packages (3.1.5)
Requirement already satisfied: python-dateutil>=2.8.2 in
.\AppData\Roaming\Python\Python314\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in
.\AppData\Roaming\Python\Python314\site-packages (from pandas) (2026.3)
Requirement already satisfied: contourpy>=1.0.1 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (4.64.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in
.\AppData\Roaming\Python\Python314\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.10.0 in
.\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (1.18.1)
Requirement already satisfied: joblib>=1.4.0 in
.\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in
.\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (2.25.0)
Requirement already satisfied: threadpoolctl>=3.5.0 in
.\AppData\Roaming\Python\Python314\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: et-xmlfile in
.\AppData\Roaming\Python\Python314\site-packages (from openpyxl) (2.0.0)
Requirement already satisfied: cloudpickle>=3.0 in
```

```
.\AppData\Roaming\Python\Python314\site-packages (from joblib>=1.4.0-
>scikitlearn) (3.1.2)
Requirement already satisfied: six>=1.5 in
.\AppData\Roaming\Python\Python314\site-packages (from pythondateutil>=2.8.2-
>pandas) (1.17.0)
```

[3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from mlxtend.frequent_patterns import apriori, association_rules
```

1 Task 1: Data Cleaning and Preparation

[4]:
```
df = pd.read_csv("Target.csv")
```

[5]:
```
df.head()
```

[5]:

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Recommendation_Frequency	Browsing_Frequency	Product_Search_Method	Search_Result_Exploration
0	2023/06/08 5:20:57 PM GMT+5:30	24	Others	Once a month	Clothing and Fashion;others	No	Rarely	Keyword	First page
1	2023/06/12 2:44:47 PM GMT+5:30	12	Male	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal…	No	Few times a month	categories	First page
2	2023/06/09 2:39:16 PM GMT+5:30	14	Prefer not to say	Few times a month	Clothing and Fashion;Home and Kitchen	No	Multiple times a day	Keyword	Multiple pages
3	2023/06/08 7:33:15 AM GMT+5:30	51	Female	Multiple times a week	Beauty and Personal Care;Clothing and Fashion;…	No	Multiple times a day	categories	First page
4	2023/06/11 11:01:04 PM GMT+5:30	46	Male	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal…	No	Few times a month	Keyword	First page

```
   Customer_Reviews_Importance … Review_Left Review_Reliability \
0                             1 …         No           Heavily
1                             5 …        Yes      Occasionally
2                             2 …        Yes            Rarely
3                             1 …         No             Never
4                             4 …         No            Rarely
  Review_Helpfulness Personalized_Recommendation_Frequency  \
0                Yes                                    4
1                Yes                                    4
2          Sometimes                                    2
3          Sometimes                                    4
4                Yes                                    3

      Recommendation_Helpfulness Rating_Accuracy Shopping_Satisfaction \
0                      Sometimes               2                     1
1                      Sometimes               5                     3
2                            Yes               3                     2
3                      Sometimes               1                     4
4                             No               2                     5

         Service_Appreciation                                Improvement_Areas \
0                   Customer service  I don't have any problem with Amazon
1                   Competitive pricesNil
2                   . I have no problem with Amazon yet. But others …
3                   Product recommendations Irrelevant product suggestions
4                   Product recommendations Add more familiar brands to the
                    list

   transaction
0       567877
1       313221
2       532959
3       310717
4       661323

[5 rows x 24 columns]
```

[6]:
```
df.shape
```

[6]: (800, 24)

[7]:
```
df.duplicated().sum()
```

[7]: np.int64(0)

[8]:
```
df.isnull().sum()
```

[8]:
```
Timestamp                             0
```

```
age                                        0
Gender                                     0
Purchase_Frequency                         0
Purchase_Categories                        0
Personalized_Recommendation_Frequency      0
Browsing_Frequency                         0
Product_Search_Method                    180
Search_Result_Exploration                  0
Customer_Reviews_Importance                0
Add_to_Cart_Browsing                       0
Cart_Completion_Frequency                  0
Cart_Abandonment_Factors                   0
Saveforlater_Frequency                     0
Review_Left                                0
Review_Reliability                         0
Review_Helpfulness                         0
Personalized_Recommendation_Frequency      0
Recommendation_Helpfulness                 0
Rating_Accuracy                            0
Shopping_Satisfaction                      0
Service_Appreciation                       0
Improvement_Areas                          0
transaction                                0
dtype: int64
```

[9]:
```
# Column names clean .
# CSV mein Personalized_Recommendation_Frequency do baar hai,
# isliye dono ko alag names denge.

recommendation_seen = 0
new_columns = []

for column in df.columns:
    column = column.strip()

    if column == "Personalized_Recommendation_Frequency":
        recommendation_seen += 1


        if recommendation_seen == 1:
            column = "Recommendation_Use_Frequency"
        else:
            column = "Recommendation_Frequency_Rating"

    new_columns.append(column)

df.columns = new_columns

print(df.columns.tolist())
```

```
['Timestamp', 'age', 'Gender', 'Purchase_Frequency', 'Purchase_Categories',
'Recommendation_Use_Frequency', 'Browsing_Frequency',
'Product_Search_Method',
```

```
'Search_Result_Exploration', 'Customer_Reviews_Importance',
'Add_to_Cart_Browsing', 'Cart_Completion_Frequency',
'Cart_Abandonment_Factors',
'Saveforlater_Frequency', 'Review_Left', 'Review_Reliability',
'Review_Helpfulness', 'Recommendation_Frequency_Rating',
'Recommendation_Helpfulness', 'Rating_Accuracy', 'Shopping_Satisfaction',
'Service_Appreciation', 'Improvement_Areas', 'transaction']
```

[10]:
```python
# Text values clean

def clean_text(value):
    if pd.isna(value):
        return np.nan

    value = str(value).strip()

    if value.lower() in ["", "nan", "nil", "."]:
        return np.nan

    return value

text_columns = df.select_dtypes(include=["object", "string"]).columns

for col in text_columns:
    df[col] = df[col].apply(clean_text)
```

[11]:
```python
# Missing Product_Search_Method handle

df["Product_Search_Method"] = (
    df["Product_Search_Method"]
    .fillna("Not specified")
    .str.title()
)

# Gender standardisation
```

```
df["Gender"] = df["Gender"].replace({
    "Others": "Other",
    "Male": "Male",
    "Female": "Female",
    "Prefer not to say": "Prefer not to say"
})
```

[12]:
```
# Exact duplicate rows remove

df = df.drop_duplicates()
```

[13]:
```
# Rating columns numeric format mein convert

rating_columns = [
    "Customer_Reviews_Importance",
    "Recommendation_Frequency_Rating",
    "Rating_Accuracy",
    "Shopping_Satisfaction"
]

for col in rating_columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")

print("Cleaned shape:", df.shape)
print(df.isnull().sum())
```

```
Cleaned shape: (800, 24)
Timestamp                          0
age                                0
Gender                             0
Purchase_Frequency                 0
Purchase_Categories                0
Recommendation_Use_Frequency       0
Browsing_Frequency                 0
Product_Search_Method              0
Search_Result_Exploration          0
Customer_Reviews_Importance        0
Add_to_Cart_Browsing               0
Cart_Completion_Frequency          0
Cart_Abandonment_Factors           0
Saveforlater_Frequency             0
Review_Left                        0
Review_Reliability                 0
Review_Helpfulness                 0
Recommendation_Frequency_Rating    0
Recommendation_Helpfulness         0
Rating_Accuracy                    0
Shopping_Satisfaction              0

Service_Appreciation              85
Improvement_Areas                 76
```

```
transaction                     0
dtype: int64
```

[14]:
```python
# Frequency values ko numeric code mein convert

purchase_map = {
    "Less than once a month ": 1,
    "Once a month": 2,
    "Few times a month ": 3,
    "Once a week": 4,
    "Multiple times a week ": 5
}

browsing_map = {
    "Rarely": 1,
    "Few times a month ": 2,
    "Few times a week ": 3,
    "Multiple times a day ": 4
}

cart_map = {
    "Never": 1,
    "Rarely": 2,
    "Sometimes": 3,
    "Often": 4,
    "Always": 5
}

df["Purchase_Frequency_Code"] = df["Purchase_Frequency"].map(purchase_map)
df["Browsing_Frequency_Code"] = df["Browsing_Frequency"].map(browsing_map)
df["Cart_Completion_Code"] = df["Cart_Completion_Frequency"].map(cart_map)
```

[15]:
```python
# Cleaned file save

df.to_csv("target_cleaned.csv", index=False)

print("Task 1 complete")
```

```
Task 1 complete
```

2 Task 2: Descriptive Behavior Analysis

[16]:
```python
# Age summary

print("Age summary:")
print(df["age"].describe())
```

```
Age summary:
count    800.000000
mean      34.737500
std       18.938306
min        3.000000
25%       18.000000
50%       34.000000
```

```
75%      50.250000
max      67.000000
Name: age, dtype: float64
```

[17]:
```python
# Gender distribution

gender_summary = df["Gender"].value_counts()
print("\nGender distribution:")
print(gender_summary)
```

```
Gender distribution:
Gender
Male                214
Female              201
Other               196
Prefer not to say   189
Name: count, dtype: int64
```

[18]:
```python
# Purchase frequency

purchase_summary = df["Purchase_Frequency"].value_counts()
print("\nPurchase frequency:")
print(purchase_summary)
```

```
Purchase frequency:
Purchase_Frequency
Multiple times a week  173
Less than once a month 168
Once a month           164
Few times a month      153
Once a week            142
Name: count, dtype: int64
```

[19]:
```python
# Purchase categories ko semicolon ke basis par split

transactions = df["Purchase_Categories"].fillna("").apply(
    lambda x: [
        item.strip().title()
        for item in str(x).split(";")
        if item.strip() and item.strip().lower() != "others"
    ]
)

category_counts = transactions.explode().value_counts()

print("Popular categories:")
print(category_counts)
```

```
Popular categories:
Purchase_Categories
```

```
Beauty And Personal Care  426
Clothing And Fashion      424
Home And Kitchen          406
Groceries And Gourmet     403
Food
Name: count, dtype: int64
```

[20]:
```
# Search method aur cart abandonment factors

print("Product search methods:")
print(df["Product_Search_Method"].value_counts())

print("\nCart abandonment factors:")
print(df["Cart_Abandonment_Factors"].value_counts())
```

```
Product search methods:
Product_Search_Method
Not Specified  180
Keyword        173
Others         158
Filter         157
Categories     132
Name: count, dtype: int64

Cart abandonment factors:
Cart_Abandonment_Factors
High shipping costs     216 others 203
Changed my mind or no longer need the item 191
Found a better price elsewhere            190
Name: count, dtype: int64
```

[21]:
```
# Mean aur median ratings

print("\nMean ratings:")
print(df[rating_columns].mean())

print("\nMedian ratings:")
print(df[rating_columns].median())
```

```
Mean ratings:
Customer_Reviews_Importance    3.00375
Recommendation_Frequency_Rating2.98125
Rating_Accuracy                3.05875
Shopping_Satisfaction          2.93500
dtype: float64

Median ratings:
Customer_Reviews_Importance    3.0
Recommendation_Frequency_Rating3.0
Rating_Accuracy                3.0
Shopping_Satisfaction          3.0
dtype: float64
```

3 Task 3: Customer Segmentation and K-Means

[22]:
```
# At-risk customer rule:
# Low satisfaction OR cart completion Never/Rarely

df["Frequent_Cart_Abandonment"] = df["Cart_Completion_Frequency"].isin(
    ["Never", "Rarely"]
)

def assign_segment(row):
    if row["Shopping_Satisfaction"] <= 2 or row["Frequent_Cart_Abandonment"]:
        return "At-Risk Customer"

    elif (
        row["Purchase_Frequency_Code"] >= 4
        and row["Shopping_Satisfaction"] >= 4
    ):
        return "Frequent Buyer"

    else:
        return "Occasional Shopper"

df["Customer_Segment"] = df.apply(assign_segment, axis=1)
```

```
print(df["Customer_Segment"].value_counts())
```

```
Customer_Segment

At-Risk Customer   533
Occasional Shopper 196
Frequent Buyer      71
Name: count, dtype: int64
```

[23]:
```
# Segment profiles

segment_profile = df.groupby("Customer_Segment").agg(
    Customers=("transaction", "count"),
    Average_Age=("age", "mean"),
    Average_Satisfaction=("Shopping_Satisfaction", "mean"),
    Average_Purchase_Frequency=("Purchase_Frequency_Code", "mean"),
    Cart_Abandonment_Rate=("Frequent_Cart_Abandonment", "mean")
).round(2)

print(segment_profile)

segment_profile.to_csv("segment_profiles.csv")
```

```
                    Customers Average_Age Average_Satisfaction \
Customer_Segment
At-Risk Customer          533       34.40                 2.39
Frequent Buyer             71       37.17                 4.51
```

```
Occasional             196       34.78                3.85
Shopper
                    Average_Purchase_Frequency Cart_Abandonment_Rate
Customer_Segment
At-Risk Customer                          2.97                  0.61
Frequent Buyer                            4.52                  0.00
Occasional Shopper                        2.48                  0.00
```

[24]:
```
# K-Means clustering

cluster_features = [
    "Purchase_Frequency_Code",
    "Browsing_Frequency_Code",
    "Cart_Completion_Code",
    "Shopping_Satisfaction",
    "Customer_Reviews_Importance",
    "Rating_Accuracy",
    "Recommendation_Frequency_Rating"
]

cluster_data = df[cluster_features].dropna()
```

```
scaler = StandardScaler()
scaled_data = scaler.fit_transform(cluster_data)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=20)

df.loc[cluster_data.index, "KMeans_Cluster"] = kmeans.fit_predict(scaled_data)

cluster_profile = df.groupby("KMeans_Cluster")[cluster_features].mean().round(2)

print(cluster_profile)

cluster_profile.to_csv("kmeans_cluster_profiles.csv")
```

```
C:\Users\Pawan\anaconda3\Lib\site-
packages\sklearn\cluster\_kmeans.py:1428: UserWarning: KMeans is known to
have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable
OMP_NUM_THREADS=4. warnings.warn(
                Purchase_Frequency_Code Browsing_Frequency_Code \
KMeans_Cluster
0.0                                2.73                    2.76
1.0                                2.99                    2.17
2.0                                3.24                    2.68
                 Cart_Completion_Code Shopping_Satisfaction \
KMeans_Cluster
0.0                              2.73                  4.36
1.0                              2.97                  2.78
2.0                              3.34                  1.70
                 Customer_Reviews_Importance Rating_Accuracy \
```

```
KMeans_Cluster
0.0                                 2.72           3.82
1.0                                 3.37           1.62
2.0                                 2.84           4.05
               Recommendation_Frequency_Rating
KMeans_Cluster
0.0    2.96 1.0   3.05
2.0                                    2.92
```

4 Task 4: Recommendation, Review and Market Basket Insights

[25]:
```
# Recommendation helpfulness numeric code

recommendation_map = {
    "No": 1,
    "Sometimes": 2,
    "Yes": 3
}

df["Recommendation_Helpfulness_Code"] = (
    df["Recommendation_Helpfulness"].map(recommendation_map)
)
```

[26]:
```
# Recommendation helpfulness aur satisfaction correlation

correlation = df[
    ["Recommendation_Helpfulness_Code", "Shopping_Satisfaction"]
].corr()

print(correlation)
```

```
                                Recommendation_Helpfulness_Code \
Recommendation_Helpfulness_Code                        1.000000
 Shopping_Satisfaction                                 0.021396

                                 Shopping_Satisfaction
Recommendation_Helpfulness_Code               0.021396
 Shopping_Satisfaction                        1.000000
```

[27]:
```
# Market Basket Analysis

all_categories = sorted({
    category
    for basket in transactions
    for category in basket
})

basket_matrix = pd.DataFrame([
    {category: category in basket for category in all_categories}
    for basket in transactions
])

frequent_itemsets = apriori(
    basket_matrix,
    min_support=0.05,
    use_colnames=True
)
```

```
rules = association_rules(
    frequent_itemsets,
    metric="lift",
    min_threshold=1
)
```

[28]:
```
# Readable category names

rules["antecedents"] = rules["antecedents"].apply(
    lambda x: ", ".join(sorted(x))
)

rules["consequents"] = rules["consequents"].apply(
    lambda x: ", ".join(sorted(x))
)

rules = rules.sort_values(
    by=["lift", "confidence", "support"],
    ascending=False
)

print(rules[
    ["antecedents", "consequents", "support", "confidence", "lift"]
].head(10))

rules.to_csv("association_rules.csv", index=False)
```

```
                                        antecedents \
25                          Clothing And Fashion
18 Beauty And Personal Care, Groceries And Gourme…
24     Groceries And Gourmet Food, Home And Kitchen 19
  Beauty And Personal Care, Clothing And Fashion
```

```
12                          Groceries And Gourmet Food, Home And Kitchen
13                          Clothing And Fashion
23     Beauty And Personal Care, Home And Kitchen 20
  Clothing And Fashion, Groceries And Gourmet Food
7                           Beauty And Personal Care, Home And Kitchen
8                           Clothing And Fashion

                                                consequents support confidence \
25 Beauty And Personal Care, Groceries And    0.0750   0.141509
Gourme…
18                         Clothing And Fashion  0.0750   0.681818
24      Beauty And Personal Care, Clothing And  0.0750   0.303030
Fashion
19        Groceries And Gourmet Food, Home And  0.0750   0.275229
Kitchen
12                         Clothing And Fashion  0.1450   0.585859
13        Groceries And Gourmet Food, Home And  0.1450   0.273585
Kitchen
23 Clothing And Fashion, Groceries And Gourmet  0.0750   0.303030
Food
20     Beauty And Personal Care, Home And Kitchen0.0750  0.271493
7                           Clothing And Fashion  0.1425     0.575758
8                           Beauty And Personal Care, Home And Kitchen
                             0.1425     0.268868

       lift 25
1.286449
18 1.286449
24 1.112038
19 1.112038
12  1.105394
13  1.105394
23 1.096942
20 1.096942
7   1.086335
8   1.086335
```

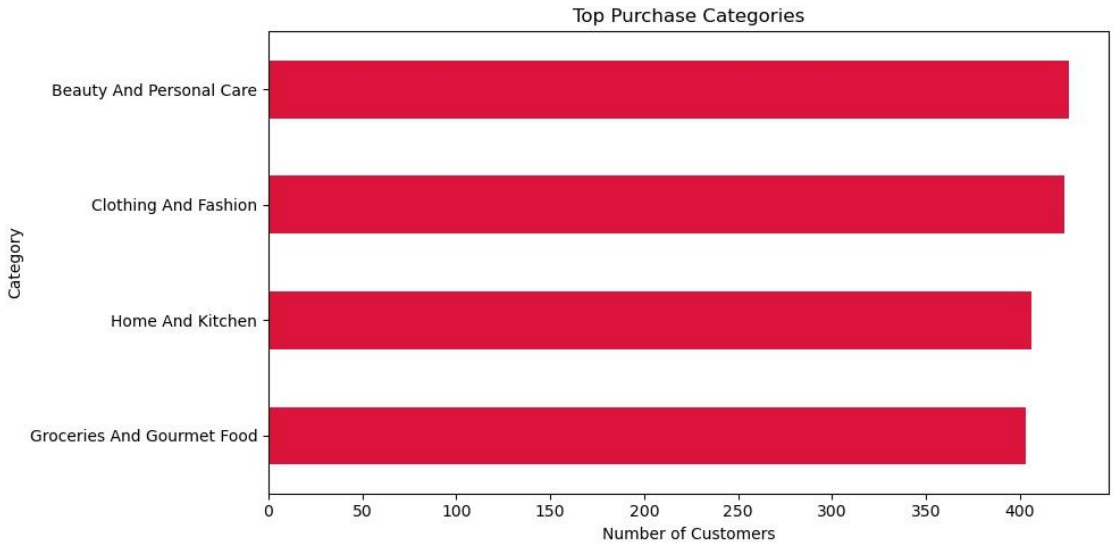
5 Task 5: Visualizations

[29]:
```
# Chart 1: Top categories

plt.figure(figsize=(10, 5))

category_counts.head(10).sort_values().plot(
    kind="barh",
    color="crimson"
)

plt.title("Top Purchase Categories")
plt.xlabel("Number of Customers")
plt.ylabel("Category")
plt.tight_layout()
plt.show()
```

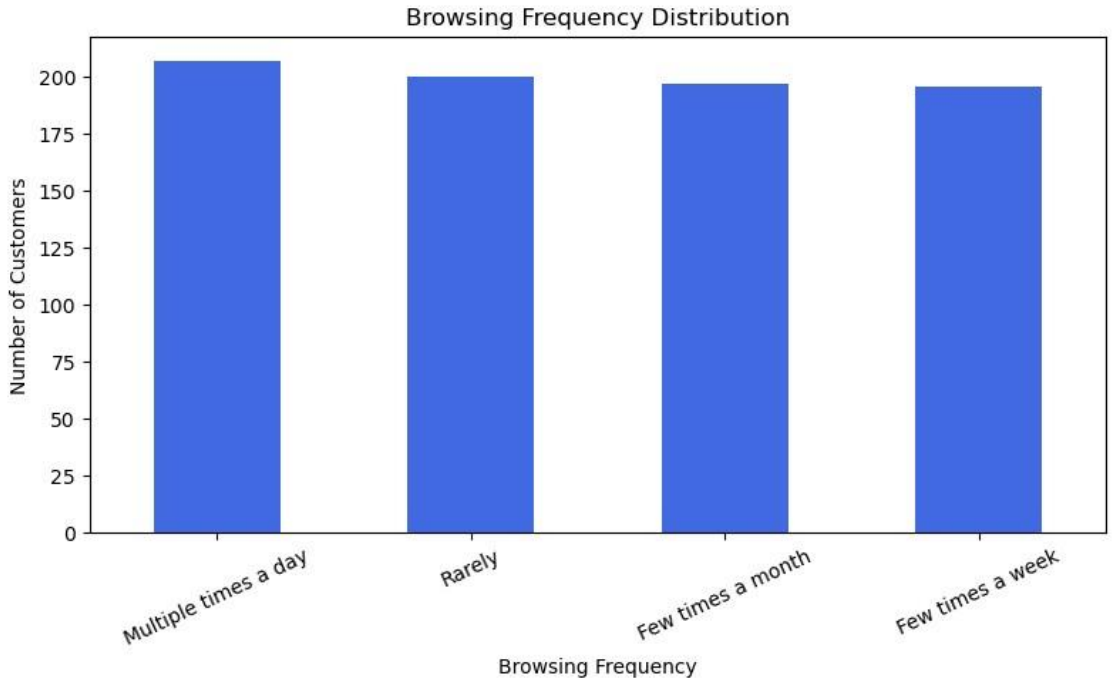


[30]:
```
# Chart 2: Browsing frequency

plt.figure(figsize=(8, 5))

df["Browsing_Frequency"].value_counts().plot(
    kind="bar",
    color="royalblue"
)

plt.title("Browsing Frequency Distribution")
plt.xlabel("Browsing Frequency")
plt.ylabel("Number of Customers")
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()
```

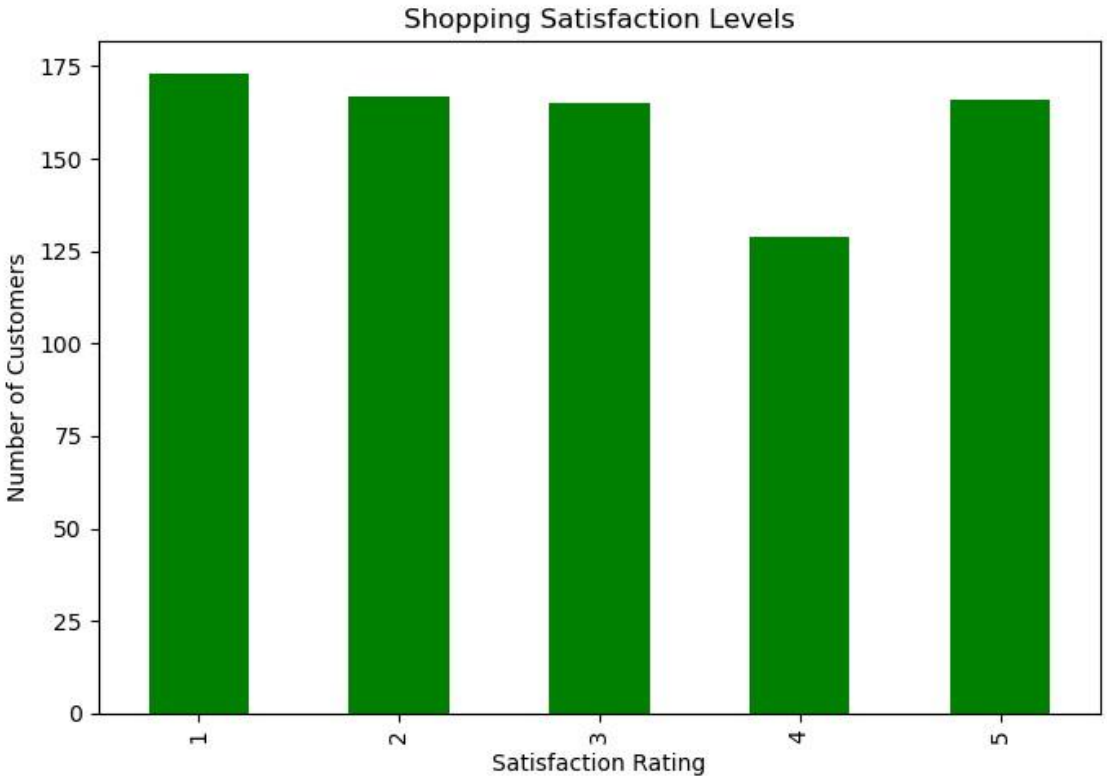


[31]:
```
# Chart 3: Satisfaction levels

plt.figure(figsize=(7, 5))

df["Shopping_Satisfaction"].value_counts().sort_index().plot(
    kind="bar",
    color="green"
)

plt.title("Shopping Satisfaction Levels")
plt.xlabel("Satisfaction Rating")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```

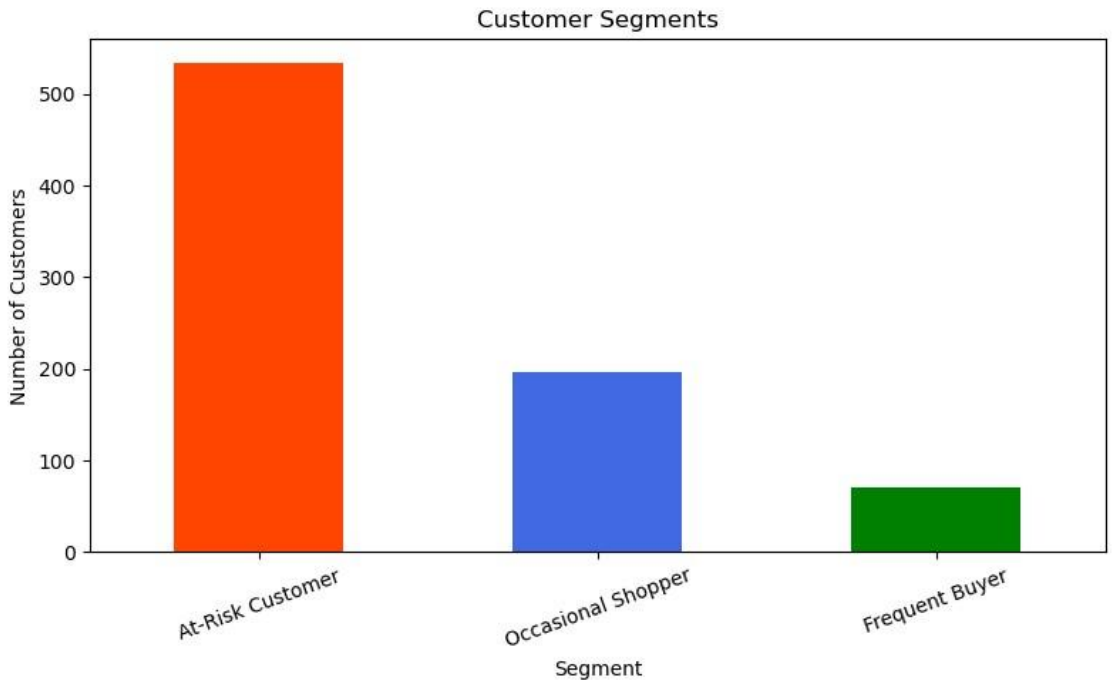


[32]:
```
# Chart 4: Customer segments

plt.figure(figsize=(8, 5))

df["Customer_Segment"].value_counts().plot(
    kind="bar",
    color=["orangered", "royalblue", "green"]
)

plt.title("Customer Segments")
plt.xlabel("Segment")
plt.ylabel("Number of Customers")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

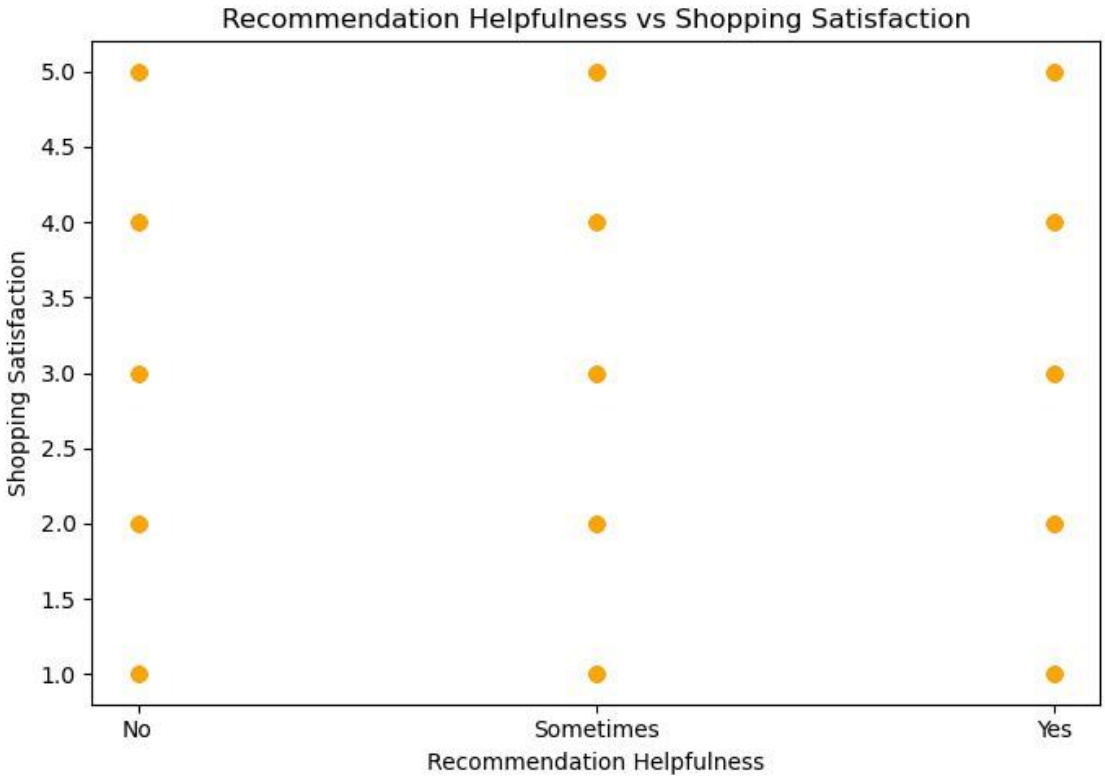


[33]:
```
# Chart 5: Recommendation helpfulness vs satisfaction

plt.figure(figsize=(7, 5))

plt.scatter(
    df["Recommendation_Helpfulness_Code"],
    df["Shopping_Satisfaction"],
    alpha=0.35,
    color="orange"
)

plt.xticks([1, 2, 3], ["No", "Sometimes", "Yes"])
plt.title("Recommendation Helpfulness vs Shopping Satisfaction ")
plt.xlabel("Recommendation Helpfulness")
plt.ylabel("Shopping Satisfaction")
plt.tight_layout()
plt.show()
```



Project Summary

Market-basket and shopping-behavior analysis was conducted on 800 customer records. The study focused on customer behavior, purchase categories, cart abandonment, satisfaction, customer segments, and product associations.

Key Insights

Average customer age: 34.7 years.
Most popular categories: Beauty & Personal Care (426), Clothing & Fashion (424), Home & Kitchen (406), and Groceries & Gourmet Food (403).
Average shopping satisfaction is low at 2.94/5.
Recommendation helpfulness has almost no relationship with satisfaction (correlation: 0.02).

Customer and Account Insights

173 customers purchase multiple times per week.
168 customers purchase less than once per month.
Frequent Buyers represent only 71 customers (8.9%), creating an opportunity for loyalty and cross-selling initiatives.
Account balance, net inflow, account ID, and financial transaction data are not available in this dataset.

Risk Insights

533 customers (66.6%) are classified as At-Risk.
The main risk drivers are low satisfaction and frequent cart abandonment.
High shipping costs are the top abandonment reason (216 customers).
Other major reasons: changed mind/no longer needed (191) and better price elsewhere (190).
"Latest Balance," "Net Inflow," and "Main Risk Account" cannot be assessed because no financial account data is included.

Segment Insights

Segment	*Customers*	*Share*	*Avg. Satisfaction*	*Key Insight*
At-Risk Customers	*533*	*66.6%*	*2.39/5*	*Priority group for retention and checkout recovery*
Occasional Shoppers	*196*	*24.5%*	*3.85/5*	*Can be converted through targeted promotions*
Frequent Buyers	*71*	*8.9%*	*4.51/5*	*Best group for loyalty and cross-selling*

Testing

Dataset size: 800 records and 24 columns.
Duplicate records: 0.
Missing Product Search Method values: 180; filled as "Not Specified."
Rule-based segmentation and K-Means clustering with 3 clusters were applied.
Further validation should include silhouette scoring for K-Means and review of invalid age entries, such as age 3.

Recommendations

Reduce or clearly communicate shipping charges to lower cart abandonment.
Run cart-recovery campaigns with reminders, discount coupons, and price-match messages.
Improve recommendation quality through A/B testing, since current recommendations do not improve satisfaction.
Create category bundles across Clothing, Beauty, Home & Kitchen, and Groceries.
Launch loyalty programs for Frequent Buyers.
Add account-level fields such as account ID, balance, deposits, withdrawals, net inflow, and risk flags for financial risk analysis.

Video explaining :-

https://www.loom.com/share/b9d09fc5bfab4ae2b96dcd016831b82c